

The Library Is the Oracle

Verification-first SoC generation from diagrams

Diagram → verified SoC: 2 model calls, 42 seconds, cycle-identical to reference

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

NXP Problem · ICLAD-DAC 2026 GenAI Chip Hackathon

The problem, and what actually scores

Given: one architecture HTML (diagrams + minimal spec), a TB skeleton fixing the top port contract, and `rtl_gen_lib` (YAML → Verilog generators). No YAML, no RTL, no text spec.

The agent must: read diagrams → infer YAML per IP → generate → stitch `secure_periph_soc` → verify.

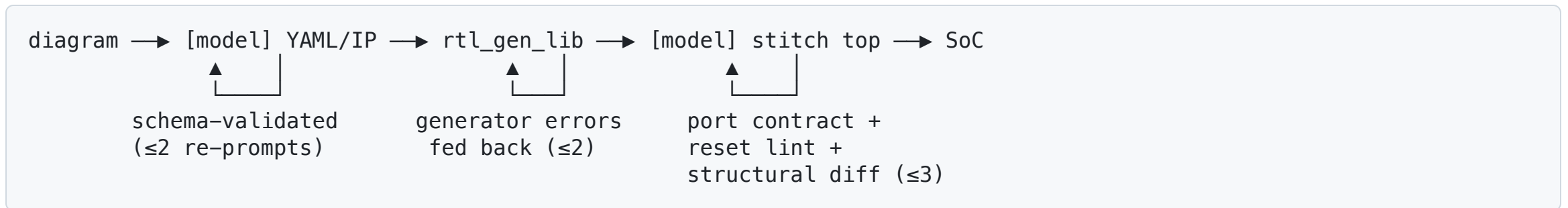
Scoring reality:

Dimension	Rule	Consequence for agent design
Correctness	hidden golden TB, <code>passed/total</code>	compile fail = 0 → never ship a broken top
Efficiency	total tokens, tiebreak	every re-prompt must earn its cost
Evaluation	runner invokes agent via <code>info.json</code> + HTTP model endpoint	contract compliance is table stakes

Thesis: constraints enforced by tooling, not prompting

Published evidence (SLDB, SpecAssess): frontier models **alter ports they were told not to touch**, and reset semantics is the **#1 silently-misspecified element**.

So every model boundary gets a deterministic gate; every failure becomes a **typed error** fed back as a bounded repair prompt:



Best-effort RTL **always ships** — a partial score beats the guaranteed 0 of a missing module.

The correctness firewall (all sabotage-validated)

Gate	Catches	Validated
port_contract	renamed / missing / wrong-width top ports vs skeleton	typed errors on all fault classes
reset_lint	por_n feeding anything but the synchronizer; split reset nets	reset = known killer
structural_diff	instance census, dangling ports, IRQ reachability, fabric hang-off, bridge bus	8/8 sabotage suite
library fixups	known non-compiling generator output (auto-patch)	behavior-neutral

Example typed error, verbatim from a live run — this text *is* the repair prompt:

```
struct-irq: i_uart.irq net 'uart_irq' never reaches u_irq_aggregator.irq_src
```

Verification depth: three independent layers

1. **30-check staged self-test TB** — reset → r/w → function → irq → watchdog → privilege;
first-failing stage localizes bugs for repair.

2. **KAT engine** — 125-command vector program, **79 known-answer checks**, replay-TB + Python comparison, **two oracles** (next slide).

3. **STG differential** — dual-SoC random-stimulus trace diff (LCG + burst schedules),
cycle-by-cycle.

The layers are complementary — measured example:

a FIFO_DEPTH 16→8 sabotage **passes the 30-check TB (30/30)** but the KAT status read catches it in 5 trace lines: `0x89` vs `0x88` — literally the `tx_full` bit.

The oracle insight: model the library, bugs included

The hidden golden TB is built on the **same generator library** — proof: we found the library watchdog's kick-reload bug (last-NBA-wins $\Rightarrow$ **kick never reloads**); a "first-principles correct" watchdog model would *fail* the golden TB.

So our reference models are transcribed from the generated RTL, statement-for-statement, with Verilog NBA semantics — quirks preserved:

- cycle-stepped Python models for **all 20 library ip_types**
- easy-8 calibrated against golden-recorded traces: **0 mismatches**
- other 12 (AXI, TileLink, AES-128, FIFOs/SRAMs): **12/12 lockstep** vs library RTL, 2000 cycles each; AES bit-exact over ~150 encryptions

Hidden testcases: an oracle that needs no golden

KAT model path: replay vectors on the candidate → predict expected values from the reference models **parameterized by the agent's own inferred YAML** → compare.

No golden RTL, no golden TB required.

Proof the oracle follows the declared design (not a hardcoded answer):

Candidate SoC	Model believes	Result
UART FIFO depth 8	<code>fifo_depth: 16</code>	FAIL 76/79 — catches the divergence
UART FIFO depth 8	<code>fifo_depth: 8</code> (plumbed)	PASS 79/79 — consistent design accepted

Runner contract: `python3 nxp_agent.py <info.json> --model <name>` — **6/6** end-to-end against a mock endpoint implementing the contest protocol. Stdlib-only.

Live with Gemini: the 5-attempt debug arc (~1 day)

#	Failure class (real)	Deterministic fix	Result
1	5/8 specs missing required params; generator failures silent	typed generator-error feedback loop	3/8 IPs
2	model repeats omissions; emits <code>---</code> mega-block	required-params schema auto-extracted from generator source ; multi-doc split	3/8
3	stitch guesses port names (<code>irq_sources</code> vs <code>irq_src</code>)	module-interface index : exact generated headers in prompt	8/8, compile fail
4	wrong <code>default_div</code> (1 vs 26); IRQ bit-order swap	library demo exemplars + doc IRQ-map extraction	30/30, KAT 68/79
5	—	—	PERFECT

Every fix is tooling that **auto-extracts from the provided materials** — nothing hand-tuned to this SoC.

The result: perfect solve, 2 calls, 42 seconds

```
[1] model returned 8 YAML spec block(s)
[2] generated 8 IP file(s): ['sys_rst_sync.v', 'ahb_brg0.v', 'apb_fab0.v', 'uart0.v',
    'gpio0.v', 'timer0.v', 'wdt0.v', 'irq_agg0.v']
[3] wrote secure_periph_soc.v (contract clean, attempt 1)
[4] GATE: PASS – 30/30 PASS
[4b] KAT(golden): PASS – 79/79
[4b] KAT(model): PASS – 79/79
[5] STG-DIFF: MATCH – 3662 cycles, 0 differing
    model calls: 2
```

- `default_div = 26` inferred correctly — **that value appears nowhere in the doc**
(recovered via library-demo exemplars: 115200 baud @ 50 MHz)
- STG: the generated SoC is **cycle-identical** to our hand-built reference

Why this generalizes: everything is extracted, nothing hardcoded

At runtime, from the *provided materials of whatever problem is given*:

- **required-params schema** — regexed from the generators' `required()` calls
- **demo exemplars** — mined from `rtl_gen_main.py --demo` specs
- **module interfaces** — parsed from the just-generated RTL headers
- **doc facts** — IRQ source map extracted from the architecture text
- structural gates activate **by role present in the design** (fabric checks only if a fabric exists, IRQ checks only if an aggregator exists...)

The same machinery re-derives itself on a medium/hard testcase with new IP types.

Medium/hard readiness

The library ships **20 generators; easy uses 8**. The other 12 are the likely medium/hard vocabulary — and they're already modeled and validated:

- AXI-Lite: crossbar (2M×3S), SRAM, DMA engine — 12/12 lockstep
- TileLink: router, network interface
- AES-128 (bit-exact), FIFOs (sync/async gray-code), SRAMs, CDC, perf counter

Known library quirks catalogued (golden-TB-relevant): AES core **ignores its encrypt input**; TL router implements **no mesh routing** (all→local). Generator `MissingParameter` errors are typed and model-directed — purpose-built repair material.

Contest contributions: 3 toolkit bugs found

1. **dma_engine** generates non-compiling RTL — `cfg_rdata` declared `output wire`, driven from `always @(*)` → elaboration error on every instance, for every team. Our agent auto-patches (behavior-neutral `wire→reg`).
2. **apb_watchdog** kick never reloads — `ctr<=ld1` overridden by later `ctr<=ctr-1` in the same `always` block (last NBA wins). Our models assert actual behavior.
3. **NVIDIA aes** ships `all_modules.v` duplicating 84 modules → masked yosys failure (invalidates naive baselines; details in our NVIDIA deck).

Found by modeling the library at statement level — depth as a by-product of the oracle strategy.

Token economics

Item	Cost
Perfect easy-tier solve	2 calls / ~40k tokens
Repair bounds	≤2 spec re-prompts, ≤2 generator-repair, ≤3 stitch
Prevention > repair	schema in the <i>first</i> prompt ended the omission loop
Thinking models	<code>thinking_budget</code> capped (unbounded: 63k thought tokens, empty output)

Every gate is **free** (deterministic, local); tokens are spent only on generation and targeted repairs with typed evidence.

By DAC (agents updatable through Jul 26)

- **K-vote spec inference** — field-level majority across independent diagram reads; disagreement → targeted re-look (defense vs consistent misreads)
- **Best-of-M stitches** — KAT-model score as the selector
- **Live-testcase runbook** — triage decision tree, budgets, panic modes (in repo)
- Medium/hard fixtures composed from the 12 modeled IPs

Summary — every claim, one command

Claim	Reproduce with
Full pipeline offline	<code>python3 nxp_agent.py --model stub</code> → 30/30, KAT 79/79 ×2
Real-model solve	<code>python3 nxp_agent.py --model vertex --deep</code>
Gate sabotage suite	<code>python3 test_structural.py</code> → 8/8
Runner contract	<code>python3 test_runner_mode.py</code> → 6/6
12 IP models vs RTL	<code>python3 test_ip_models.py</code> → 12/12

Repo: github.com/chelsea85/chip-convergence-iclad26 (fresh-clone verified)

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com